

CS3300 Introduction to Software Engineering

# Lecture 08: Tools of the Trade #4

Spring, Spring Boot

Dr. Nimisha Roy ▶ [nroy9@gatech.edu](mailto:nroy9@gatech.edu)

# Contents

- What Maven solves - and what it doesn't
- Frameworks
- Spring
  - IoC and Dependency Injection
- Spring Boot
  - Starter Dependencies & Auto-Configuration
- Demo: Building a REST API

# Maven vs Framework

| Maven                      | Framework                                |
|----------------------------|------------------------------------------|
| Manages dependencies       | Provides application structure           |
| Builds/packages code       | Provides reusable application components |
| Runs tests/plugins         | Handles common application concerns      |
| Standard project lifecycle | Defines conventions and control flow     |

# Frameworks

- Framework is a predefined set of tools, libraries, best practices, and conventions that helps developers create applications more efficiently and effectively.
- Provides a foundation on which developers can build programs for a specific platform.

Provides a  
consistent  
structure for  
collaboration

Reusable Code

Includes Best  
practices for  
foundational  
elements

Extensions and  
Plugins

Inversion of  
Control

Learning Curve

# Java Frameworks

- Java framework is a body of reusable prewritten code acting as templates used by developers to create apps using the Java programming language

## Web frameworks

- SpringBoot, Vaadin, JavaServer Faces, JHipster

## Data Access frameworks

- Hibernate, Java Persistence API, MyBatis

## Microservices

- Spring Cloud, MicroProfile, Quarkus

## Big Data

- Apache Kafka, Apache Spark, Apache Hadoop

## Testing

- Junit, TestNG, Mockito, Spring Test

## Security

- Spring Security

# Framework *vs.* API

## **API**

Interface through which software components communicate

Last week: REST endpoints such as GET /api/products

## **Framework**

Provides structure and reusable infrastructure for building an application

Can expose APIs and provide APIs for developers to use

# Spring

- A Java framework for building modular applications.
- Spring provides infrastructure for common application concerns such as:
  - Web applications and REST APIs
  - Dependency management between application components
  - Data access
  - Security
  - Testing

**Core idea:** Spring can create and manage application objects for us.

# Inversion of Control (IoC)

Normally:

**Your code → creates objects → calls other code**

With IoC:

- **Framework → creates/manages components → calls your code**

Spring's **IoC container** manages application components and their dependencies.

# Inversion of Control (IoC)

Example of highly coupled classes

```
public class DataAccess
{
    public DataAccess()
    {
    }

    public string GetCustomerName(int id) {
        return "Dummy Customer Name"; // get it
    }
}
```

```
public class CustomerBusinessLogic
{
    DataAccess _dataAccess;

    public CustomerBusinessLogic()
    {
        _dataAccess = new DataAccess();
    }

    public string GetCustomerName(int id)
    {
        return _dataAccess.GetCustomerName(id);
    }
}
```

- *CustomerBusinessLogic* and *DataAccess* classes are tightly coupled. Changes in the *DataAccess* class will lead to changes in the *CustomerBusinessLogic* class. For example, if we add, remove or rename any method in the *DataAccess* class then we need to change the *CustomerBusinessLogic* class accordingly.
- The *CustomerBusinessLogic* class creates an object of the *DataAccess* class using the **new** keyword. There may be multiple classes which use the *DataAccess* class and create its objects. So, if you change the name of the class, then you need to find all the places in your source code where you created objects of *DataAccess* and make the changes throughout the code.
- Because the *CustomerBusinessLogic* class creates an object of the concrete *DataAccess* class, it cannot be tested independently (TDD). The *DataAccess* class cannot be replaced with a mock class.

# Inversion of Control (IoC)

Using Factory pattern of the IoC principle => Loosely coupled design

```
public class CustomerBusinessLogic
{
    public CustomerBusinessLogic()
    {
    }

    public string GetCustomerName(int id)
    {
        DataAccess _dataAccess = DataAccessFactory.GetDataAccessObj();

        return _dataAccess.GetCustomerName(id);
    }
}
```

```
public class DataAccess
{
    public DataAccess()
    {
    }

    public string GetCustomerName(int id) {
        return "Dummy Customer Name"; // get it
    }
}
```

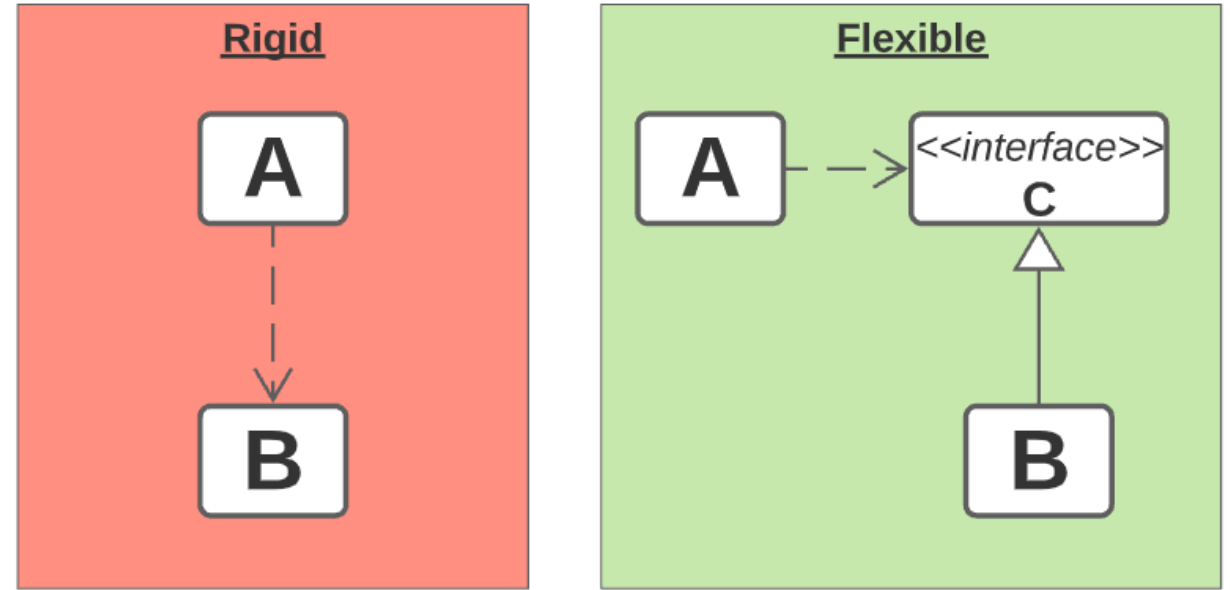
```
public class DataAccessFactory
{
    public static DataAccess GetDataAccessObj()
    {
        return new DataAccess();
    }
}
```

The *CustomerBusinessLogic* class uses the *DataAccessFactory.GetDataAccessObj()* method to get an object of the *DataAccess* class instead of creating it using the new keyword. Thus, we have inverted the control of creating an object of a dependent class from the *CustomerBusinessLogic* class to the *DataAccessFactory* class.

**first step** towards achieving fully loose coupled design.

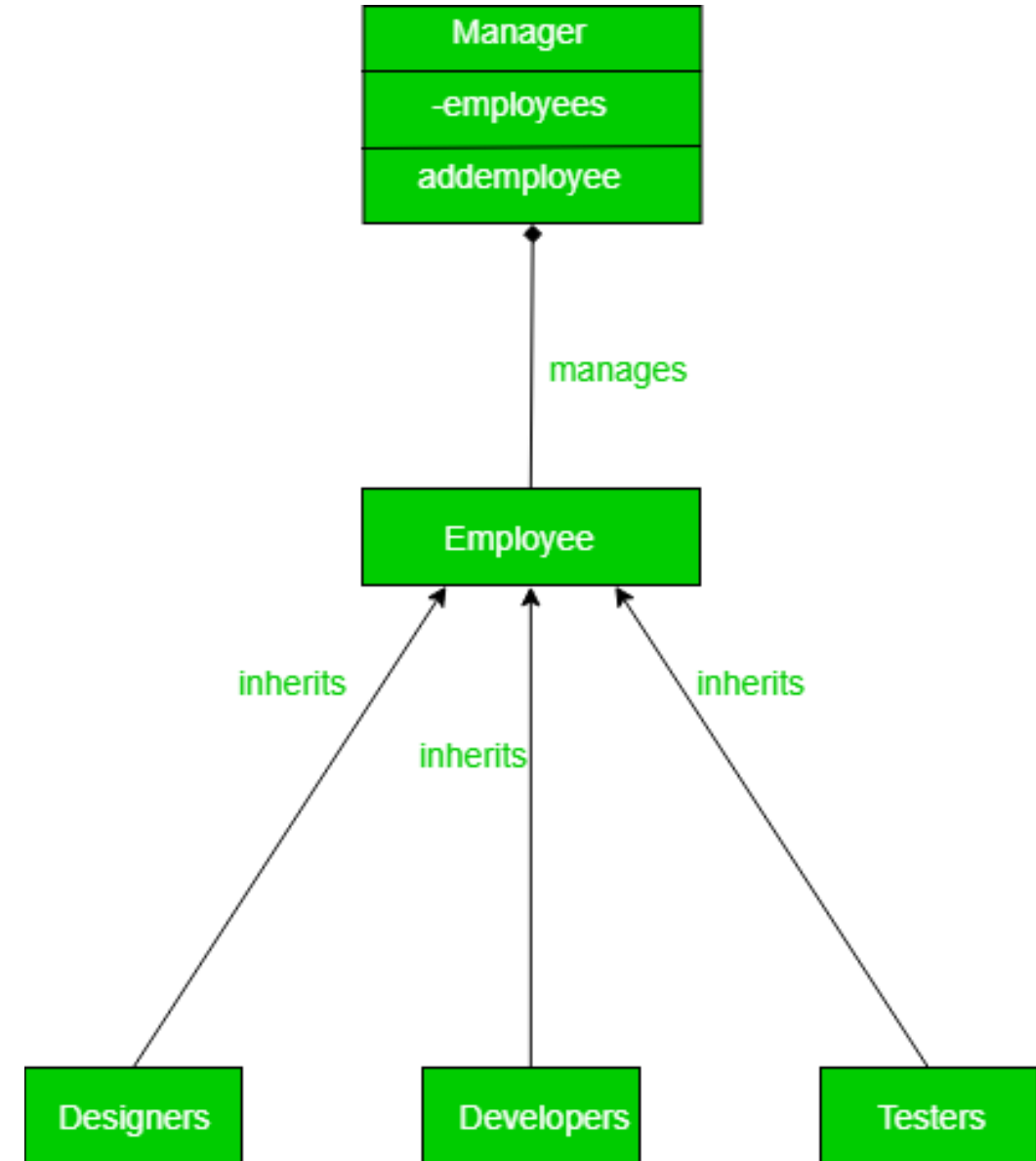
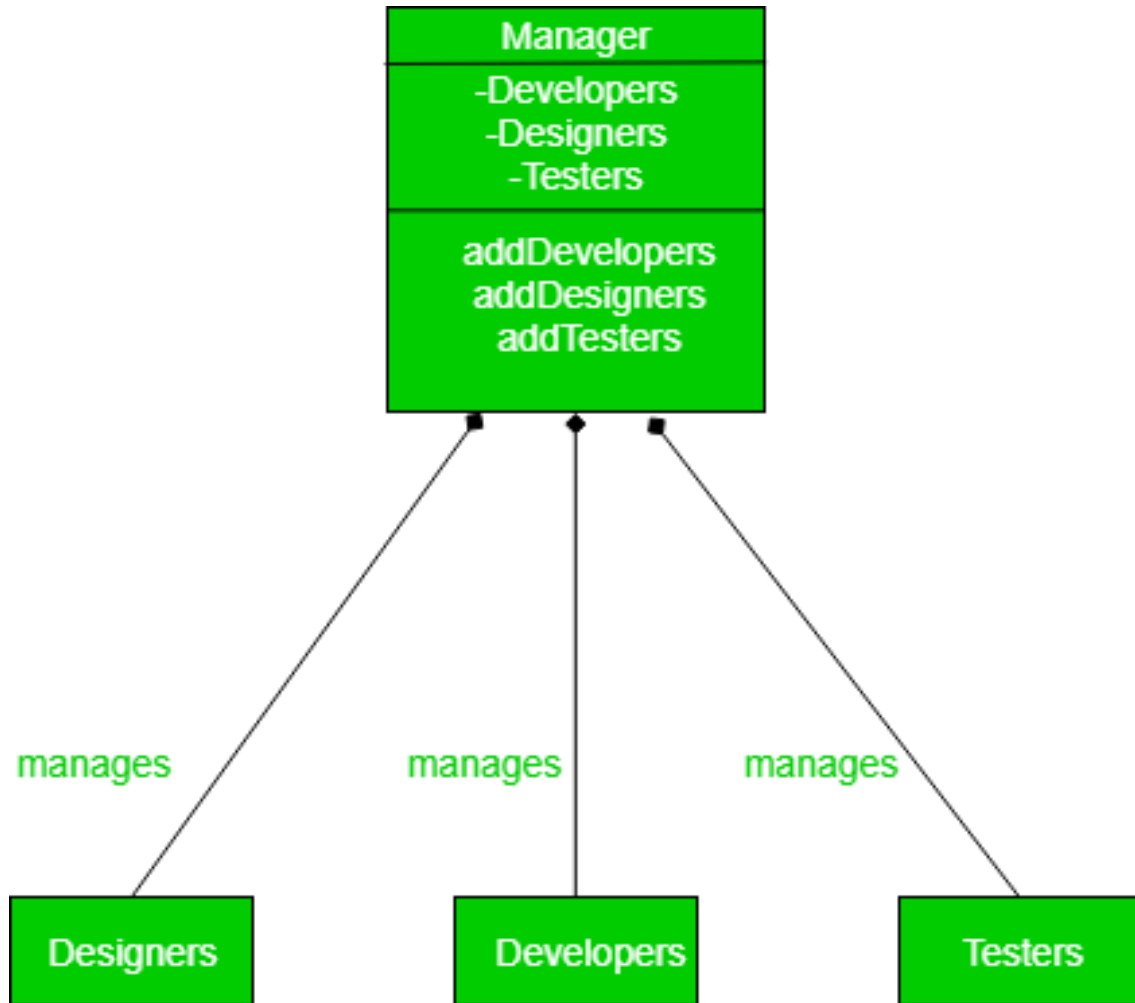
# Dependency Inversion Principle (DIP)

- A high-level module (depends on other modules) should not depend on low-level modules (*DataAccess Class*). Both should depend on abstraction.
- Abstractions should not depend on details. Details should depend on abstractions



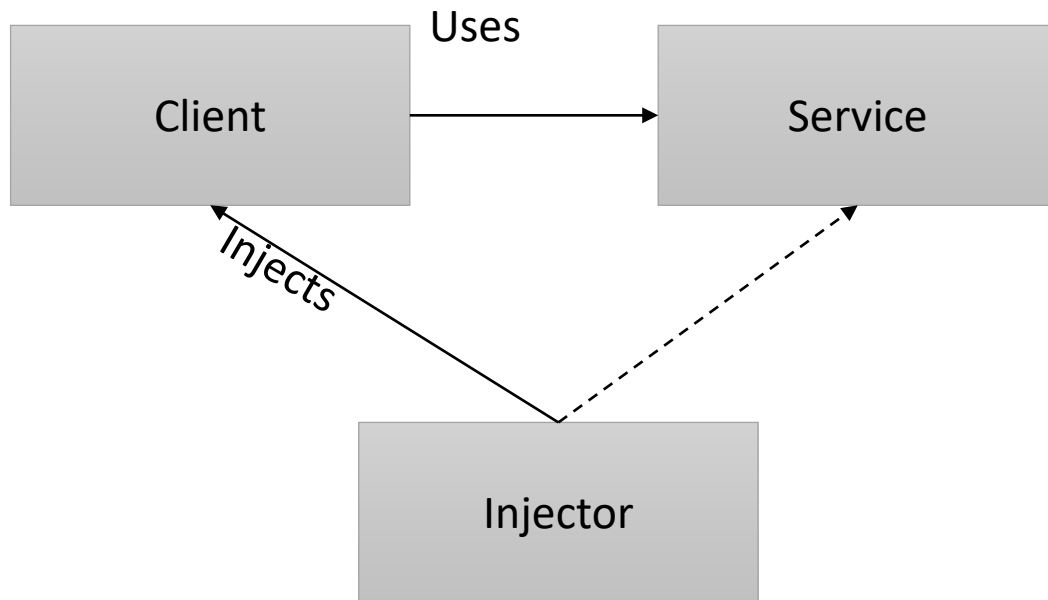
Abstraction in OOPS means to create an interface or an abstract class which is non-concrete. This means we cannot create an object of an interface or an abstract class.

# Dependency Inversion Principle (DIP)



# Dependency Injection (DI)

- Dependency Injection (DI) is a design pattern used to implement IoC. It allows the creation of dependent objects outside of a class and provides those objects to a class through different ways.
- Increases possibility to reuse classes and test independently of other classes while unit testing
- DI pattern includes 3 class types:



Injector class creates an object of service class, injects it to a client object. Hence, separates the responsibility of creating an object of service class out of the client class.

## Benefits of using DI

- Helps in Unit testing.
- Boiler plate code is reduced, as initializing of dependencies is done by the injector component.
- Extending the application becomes easier.
- Helps to enable loose coupling, which is important in application programming.

# Without DI example

```
// Engine class
class Engine {
    public void start() {
        System.out.println("Engine started.");
    }
}
```

## Problems with This Approach:

- Tight Coupling:** The Car class is tightly coupled with the Engine class. If you want to change the engine type (e.g., to a DieselEngine), you must modify the Car class code.
- Difficult to Test:** You cannot easily replace the Engine with a mock or stub for testing purposes.
- Low Flexibility:** Every change in the Engine class or its behavior requires changes in the Car class.

```
// Car class without Dependency Injection
class Car {
    private Engine engine;

    public Car() {
        // The Car class creates its own Engine instance
        this.engine = new Engine();
    }

    public void start() {
        engine.start();
        System.out.println("Car started.");
    }
}

// Usage
public class Main {
    public static void main(String[] args) {
        Car car = new Car(); // Car creates its own dependencies
        car.start();
    }
}
```

# With DI example

```
// Service class
class Engine {
    public void start() {
        System.out.println("Engine started.");
    }
}
```

```
// Injector class
class CarInjector {
    public static Car getCar() {
        // Create the service instance (Engine)
        Engine engine = new Engine();

        // Inject the service into the client and return the client
        return new Car(engine);
    }
}
```

```
// Client class
class Car {
    private Engine engine; // Dependency

    // Constructor Injection: The Engine is injected via the constructor
    public Car(Engine engine) {
        this.engine = engine;
    }

    public void start() {
        engine.start(); // Using the injected dependency
        System.out.println("Car started.");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        // Use the injector to get the client with the service injected
        Car car = CarInjector.getCar();
        car.start();
    }
}
```

Dependency Injection means a class receives the objects it depends on instead of creating those objects itself.

# With DI example

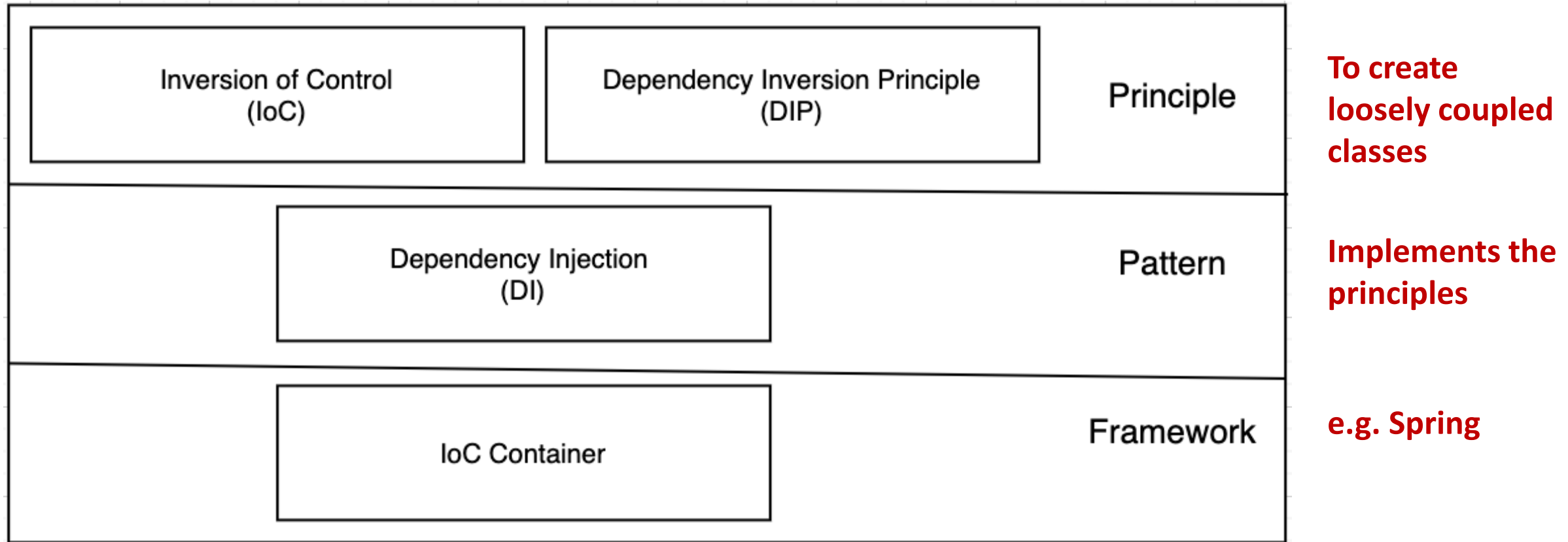
## Benefits of This Approach:

- **Loose Coupling:** The Car class is not tightly coupled to the Engine class. Any implementation of Engine (e.g., DieselEngine, ElectricEngine) can be injected.

**Testability:** Easily test the Car class by injecting mock objects or stubs of the Engine. I can replace the real dependency with a controlled test dependency. That's one reason dependency injection makes software much easier to unit test.

- **Single Responsibility Principle:** The CarInjector class handles the creation and injection of dependencies, leaving the Car class focused on its core functionality.

# IoC, DIP, DI



When Spring creates and manages one of these objects, we call that object a **Spring bean**. For our purposes:

**Bean = an object managed by Spring's IoC container.**

# Spring Boot

Spring Boot builds on the Spring Framework and reduces the setup required to create and run Spring applications.

## **What does Spring Boot add?**

### **Starter Dependencies**

Convenient sets of dependencies for common application types. Example: spring-boot-starter-web. Maven still resolves and manages the dependencies

### **Auto-Configuration**

Configures the application based on the dependencies and settings present. Provides sensible defaults that can be overridden

### **Embedded Server**

Web server such as Tomcat runs with the application. No separate server installation/deployment required

**Spring = application framework. Spring Boot = easier setup, configuration, and execution of Spring applications**

# DEMO TIME!!

- Create a simple web app using SpringBoot
- We needed Express in Node.js to establish the server. Spring Boot has Tomcat server embedded– very convenient

## **WHAT IF WE HAD TO CREATE THE SAME APP WITHOUT SPRING BOOT**

- We have Spring framework
- Setup and manage all maven dependencies and versions in pom.xml manually – very extensive
- Define Web.xml file to configure web related front controller (if war)
- Define a Spring context XML file to define component scans
- Install Tomcat or configure Tomcat maven plugin
- Deploy and run application IN TOMCAT

# Spring Boot Starter Projects

- Goal is to help you get a project up and running quickly
  - Web application: Spring-Boot-Starter-Web
  - REST API: Spring-Boot-Starter-Web
  - Talk to database using JPA- Spring-Boot-Starter-Data-JPA
  - Talk to database using JDBC- Spring-Boot-Starter-JDBC
  - Secure web application- Spring-Boot-Starter-Security
- Manage list of maven dependencies & versions for different apps:
  - Spring-Boot-Starter-Web: Frameworks needed by typical web applications. Spring-webmvc, spring-web, spring-boot-starter-tomcat, spring-boot-starter-json

# Spring Boot Auto Configuration

- Starter defines dependencies
- Auto configurations provides basic configuration to run application using frameworks defined in your maven dependencies
- Decided based on:
  - Which frameworks are in class path?
  - What is the existing configuration? (*Maven Dependencies – `springframework.boot.autoconfigure` – classes that will be checked*)
  - Type *`logging.level.org.springframework=DEBUG`* in *`application.properties`* to see what is being auto-configured.